COMPONENT BASED SOFTWARE ENGINEERING

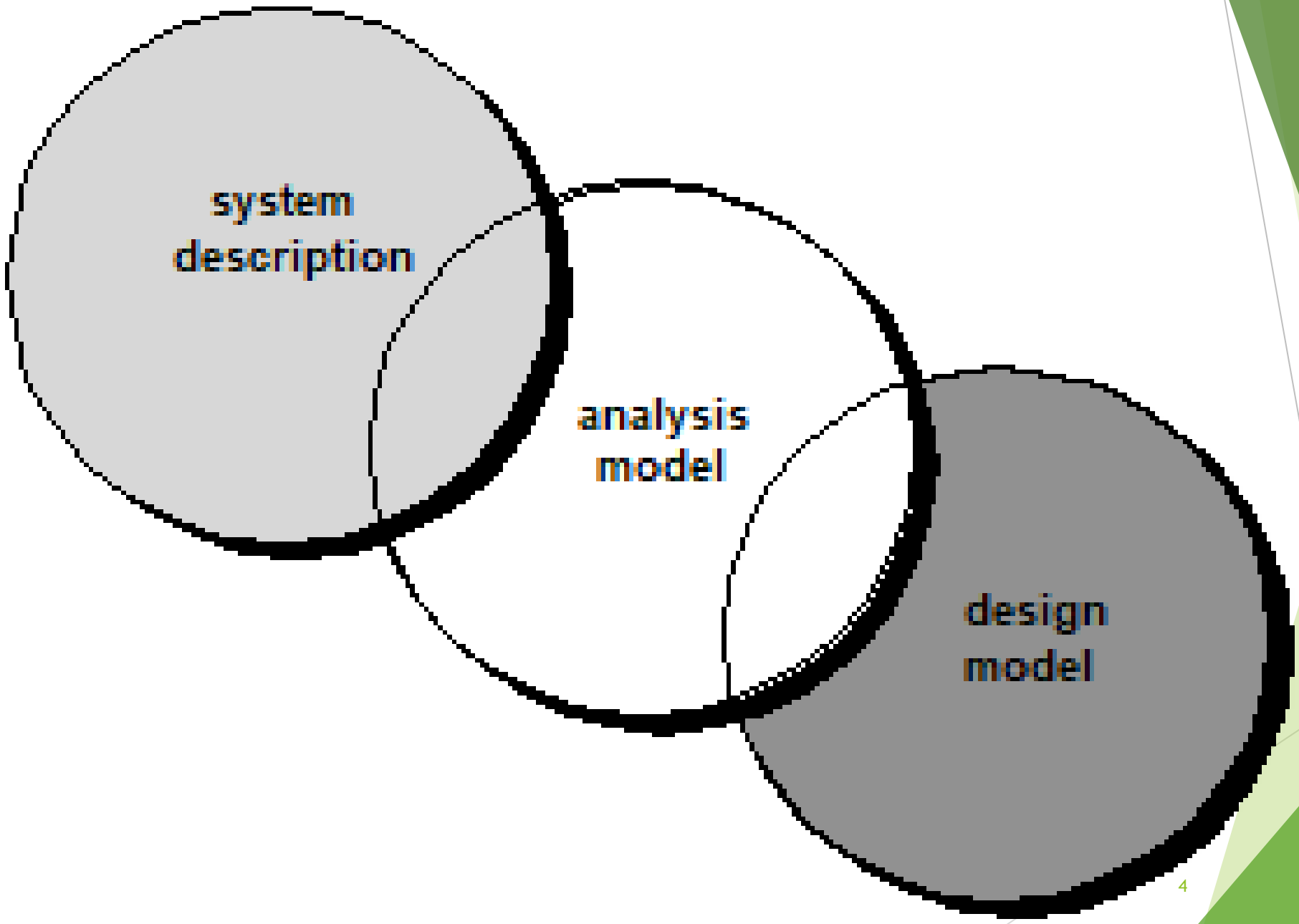
Component Modelling

Requirements Analysis

- *Requirement Analysis* results in the specification of software's operational characteristics; indicates software's interface with other system element; and establishes constraints that software must need.
- Throughout analysis modeling the software engineer's primary focus is on *what* not *how*.
- RA allows the software engineer (called an *analyst* or *modeler* in this role) to:
  - Elaborate on basic requirements established during earlier requirement engineering tasks
  - Build models that depict user scenarios, functional activities, problem classes and their relationships, system and class behavior, and the flow of data as it is transformed.

Overall Objectives and Philosophy

- The analysis model must achieve three primary objectives:
  - To describe what the customer requires
  - To establish a basis for the creation of a software design, and
  - To define a set of requirements that can be validated once the software is built.
- The analysis model bridges the gap between a system-level description that describes overall functionality as it is achieved by applying software, hardware, data, human, and other system elements and a software design that describes the software application architecture.


system
description
analysis
model
design
model

Analysis Rules of Thumb

- The model should focus on requirements that are visible within the problem or business domain. The level of abstraction should be relatively high.
- Each element of the analysis model should add to an overall understanding of software requirements and provide insight into the information domain, function and behavior of the system.
- Delay consideration of infrastructure and other non-functional models until design.
  - For example, a database may be required, but the classes necessary to implement it, the functions required to access it, and the behavior that will be exhibited as it is used should be considered only after problem domain analysis has been completed.

Cont.:Analysis Rules of Thumb

- Minimize coupling throughout the system.
  - The level of interconnectedness between classes and functions should be reduced to a minimum.
- Be certain that the analysis model provides value to all stakeholders.
  - Each constituency has its own use for the model.
- Keep the model as simple as it can be.
  - Ex: Don’t add additional diagrams when they provide no new information.
  - Only modeling elements that have values should be implemented.

Domain Analysis

- Software domain analysis is the identification, analysis, and specification of common requirements from a specific application domain, typically for reuse on multiple projects within that application domain . . . [Object-oriented domain analysis is] the identification, analysis, and specification of common, reusable capabilities within a specific application domain, in terms of common objects, classes, subassemblies, and frameworks . . . ***Donald Firesmith***
- Define the domain to be investigated.
- Collect a representative sample of applications in the domain.
- Analyze each application in the sample.
- Develop an analysis model for the objects

Analysis Modeling Concepts and Approaches

- One view of analysis modeling, called ***structural analysis***, considers data and the processes that transform the data as separate entities.
- Data objects are modeled in a way that defines attributes and relationships.
- Processes that manipulate data objects are in a manner that shows how they transform data as data objects flow through the system.
- A second approach to analysis modeling called ***object-oriented analysis*** focuses on the definition of classes and the manner in which they collaborate with one another to affect customer requirements. UML and the Unified Process are predominantly Object Oriented.

Data Modeling Concepts

1. **Data Objects:** is a representation of almost any composite information that must be processed by software. By composite, we mean something that has a number of different properties and attributes.
2. **Data Attributes:** *define* the properties of a data object and take one of three different characteristics. They can be used to:
   - Name an instance of the data object.
   - Describe the instance, or
   - Make reference to another instance in another table.
3. ***Relationships:*** Indicates “connectedness”; a "fact" that must be "remembered" by the system and cannot or is not computed or derived mechanically
   - several instances of a relationship can exist
   - objects can be related in many different ways

- We can define a set of object/relationship pairs that define the relevant relationships. For example:
- A person *owns* a car. A person is *insured to drive* a car.

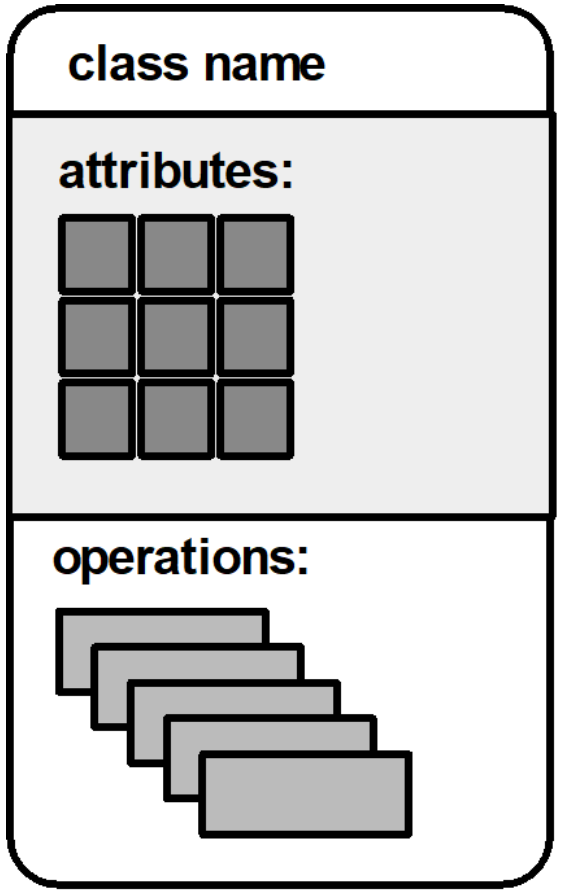
Object-Oriented Analysis

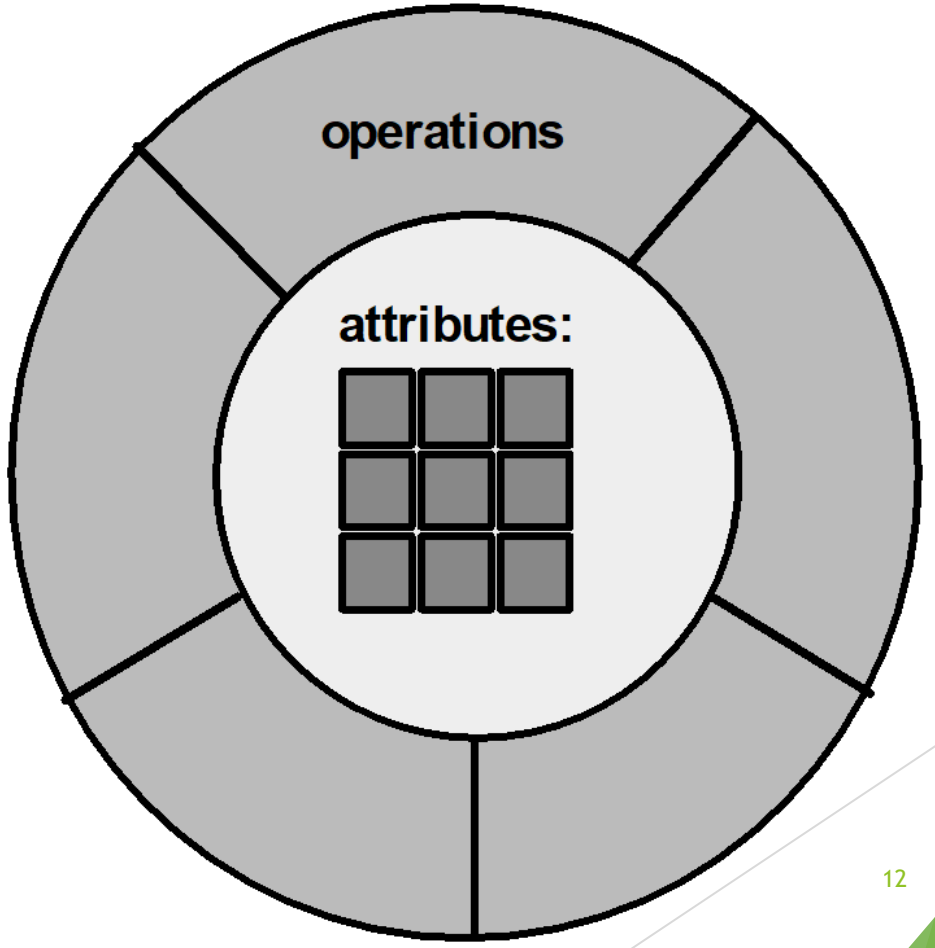
- The intent of Object-Oriented Analysis (OOA) is to define all classes (and the relationships and behavior associated with them that are relevant to the problem to be solved.
- To accomplish this, a number of tasks must occur:
  1. Basic user requirements must be communicated between the customer and the software engineer.
  2. Classes must be defined.
  3. A class hierarchy is defined
  4. Object-to-object relationships should be represented.
  5. Object behavior must be modeled.
  6. 1 – 5 are repeated iteratively until the model is complete.

Cont.:

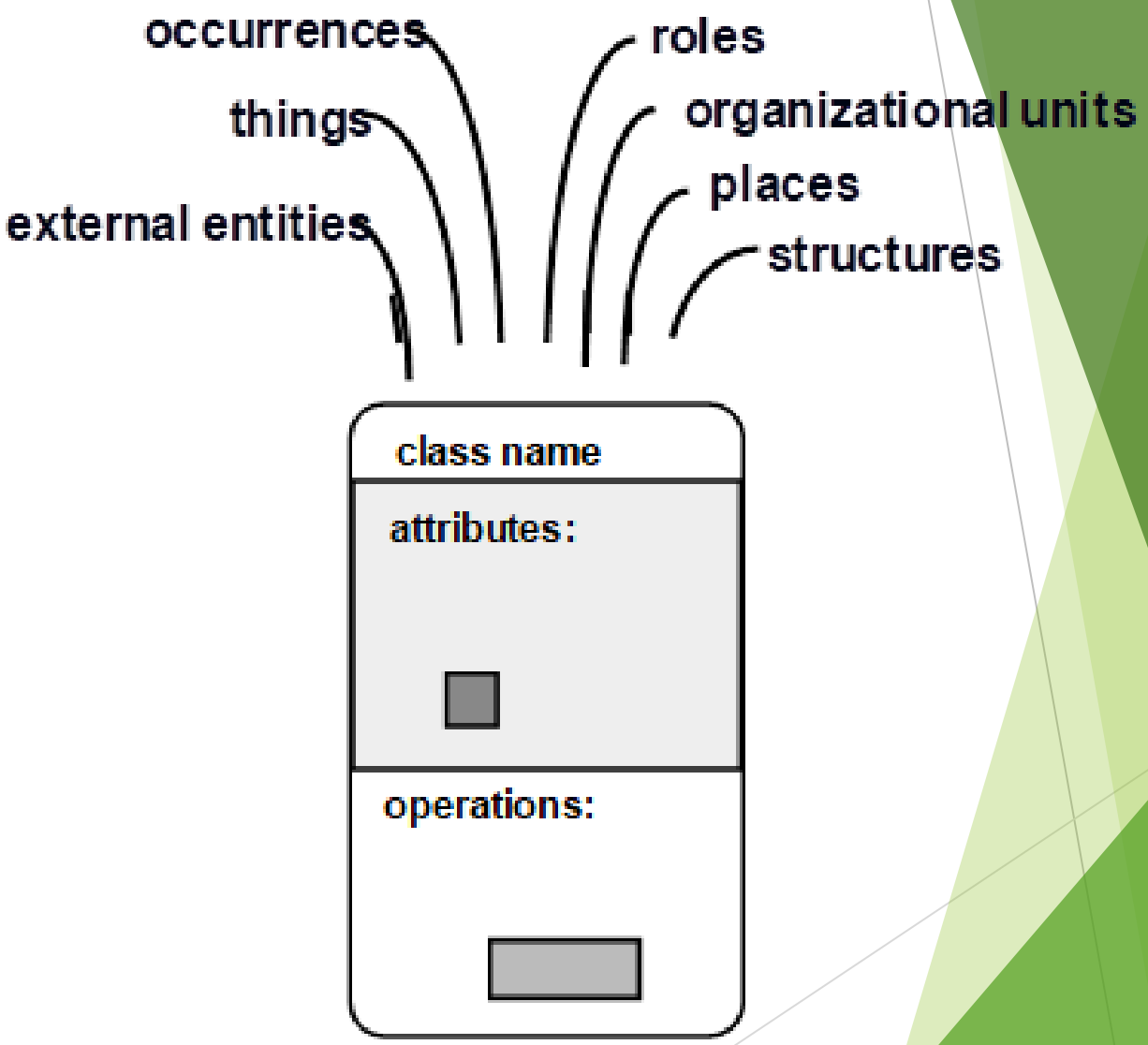
- OOA builds a class-oriented model that relies on an understanding of OO concepts.
  - Classes and objects
  - Attributes and operations
  - Encapsulation and instantiation
  - Inheritance
- Object-Oriented thinking begins with the definition of a class, often defined as:
  - template
  - generalized description
  - “blueprint” ... describing a collection of similar items
  - a metaclass (also called a superclass) establishes a hierarchy of classes once a class of items is defined, a specific instance of the class can be identified.

Building a class





Building a class



Scenario-Based Modeling

- "[Use-cases] are simply an aid to defining what exists outside the system (actors) and what should be performed by the system (use-cases)." Ivar Jacobson
- The concept is relatively easy to understand- describe a specific usage scenario in straightforward language from the point of view of a defined actor.

(1) Writing Use-Cases

(1) What should we write about? : Inception and elicitation provide us the information we need to begin writing use cases.

(2) How much should we write about it?

(3) How detailed should we make our description?

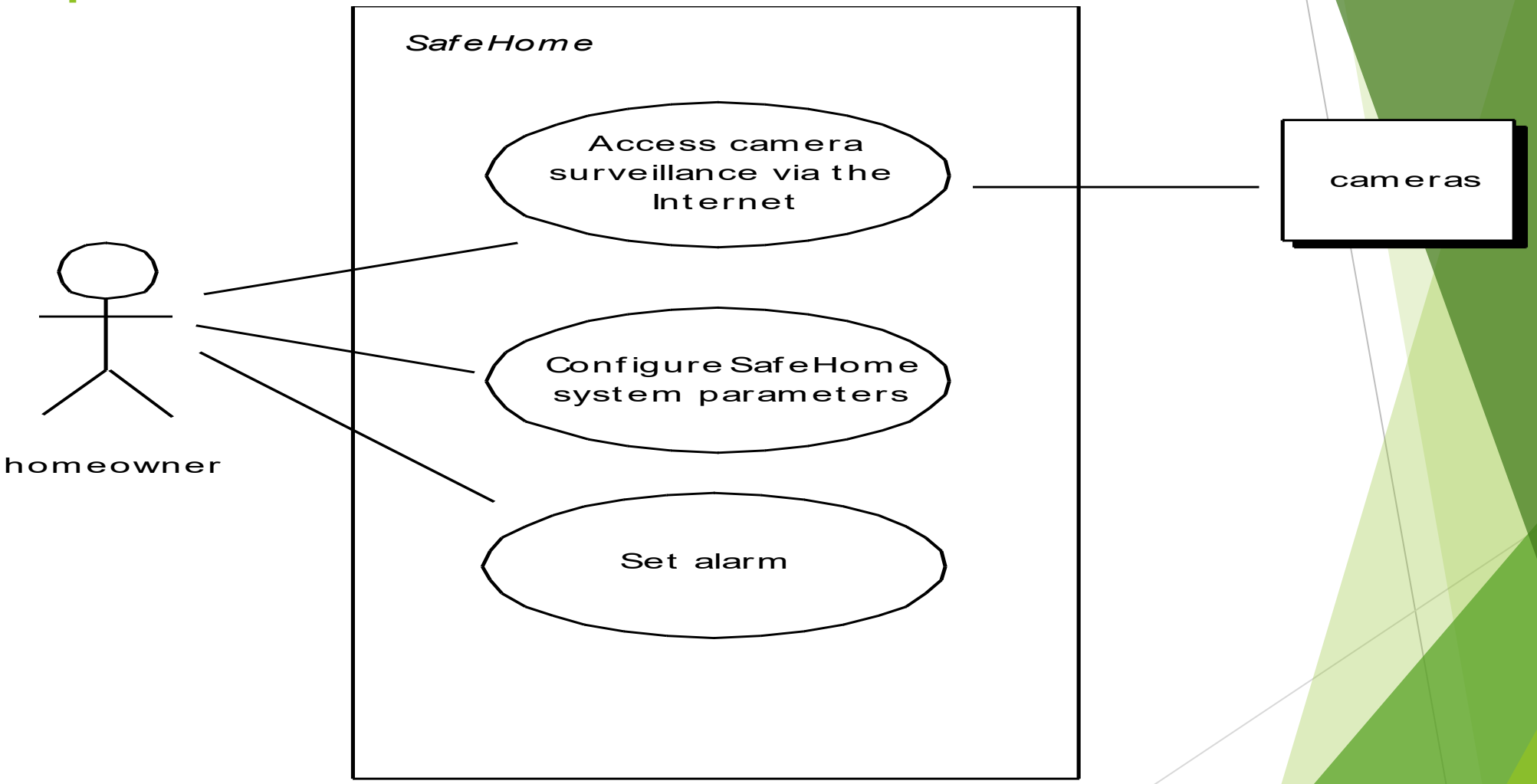
(4) How should we organize the description?

- Use Cases:
  - A scenario that describes a “thread of usage” for a system.
  - ***Actors*** represent roles people or devices play as the system functions.
  - ***Users*** can play a number of different roles for a given scenario.

(2) Developing an Activity Diagram-Nutshel

- What are the main tasks or functions that are performed by the actor?
- What system information will the actor acquire, produce or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

Example


SafeHome
Access camera surveillance via the Internet
Configure SafeHome system parameters
Set alarm
homeowner
cameras

(3) Swimlane Diagrams

- The UML *swimlane diagram* is a useful variation of the activity diagram and allows the modeler to represent the flow of activities described by the user-case and at the same time indicate which actor or analysis class has responsibility for the action described by an activity rectangle.
- Responsibilities are represented as parallel segments that divide the diagram vertically, like the lanes in a swimming pool.

(4) Flow-Oriented Modeling

- Represents how data objects are transformed as they move through the system.
- A *data flow diagram* (DFD) is the diagrammatic form that is used to complement UML diagrams.
- Considered by many to be an ‘old school’ approach, flow-oriented modeling continues to provide a view of the system that is unique.
- The DFD takes an input-process-output insight into system requirements and flow.
- Data objects are represented by labeled arrows and transformations are represented by circles (called *bubbles)*.

Creating a Data Flow Model

- The DFD diagram enables the software engineer to develop models of the information domain and functional domain at the same time.
- As the DFD is refined into greater levels of detail, the analyst performs an implicit functional decomposition of the system.
- **Guidelines**:
  - The level 0 data DFD should depict the software/system as a single bubble.
  - Primary I/O should be carefully noted.
  - Refinement should begin by isolating candidate processes, data objects, and data stores.
  - All arrows and bubbles should be labeled with meaningful names.
  - Information flow continuity must be maintained from level to level.
  - One bubble at one time should be refined.
- Information continuity must be maintained at each level as DFD level is refined. This mean that input and output at one level must be the same as input and output at a refined level.

(5) Class-Based Modeling

- **Identifying Analysis Classes**
- Identify analysis classes by examining the problem statement
- Use a “grammatical parse” to isolate potential classes
- Identify the attributes of each class
- Identify operations that manipulate the attributes
- *Analysis Classes* manifest themselves in one of the following ways:

Cont.:

- *External entities* (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
- *Things* (e.g., reports, displays, letters, signals) that are part of the information domain for the problem.
- *Occurrences or events* (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
- *Roles* (e.g., manager, engineer, salesperson) played by people who interact with the system.
- *Organizational units* (e.g., division, group, and team) that are relevant to an application.
- *Places* (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
- *Structures* (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or related classes of objects.

(6) Creating a Behavioral Model

- The behavioral model indicates how software will respond to external events or stimuli. To create the model, the analyst must perform the following steps:

1. Evaluate all use-cases to fully understand the sequence of interaction within the system.
2. Identify events that drive the interaction sequence and understand how these events relate to specific objects.
3. Create a sequence for each use-case.
4. Build a state diagram for the system.
5. Review the behavioral model to verify accuracy and consistency

State Representations

- In the context of behavioral modeling, two different characterizations of states must be considered:
  - the state of each class as the system performs its function and
  - the state of the system as observed from the outside as the system performs its function
- The state of a class takes on both passive and active characteristics.
  - A *passive state* is simply the current status of all of an object's attributes.
  - The *active state* of an object indicates the current status of the object as it undergoes a continuing transformation or processing.

The States of a System

- State—a set of observable circumstances that characterizes the behavior of a system at a given time
- State transition—the movement from one state to another
- Event—an occurrence that causes the system to exhibit some predictable form of behavior
- Action—process that occurs as a consequence of making a transition

Components Modelling

Using UML

UML2 Structure diagrams

- Structure diagrams show the static structure of the system and its parts on different abstraction and implementation levels and how they are related to each other.
- The elements in a structure diagram represent the meaningful concepts of a system, and may include abstract, real world and implementation concepts, there are seven types of structure diagram as well as behaviour representation diagrams
- Behavior representation diagrams show the dynamic behavior of the objects in a system, which can be described as a series of changes to the system over time, there are seven types of behavior diagrams

Examples

Implementation representation

- Class Diagram
- Component Diagram
- Deployment Diagram
- Object Diagram
- Package Diagram
- Composite Structure Diagram
- Profile Diagram

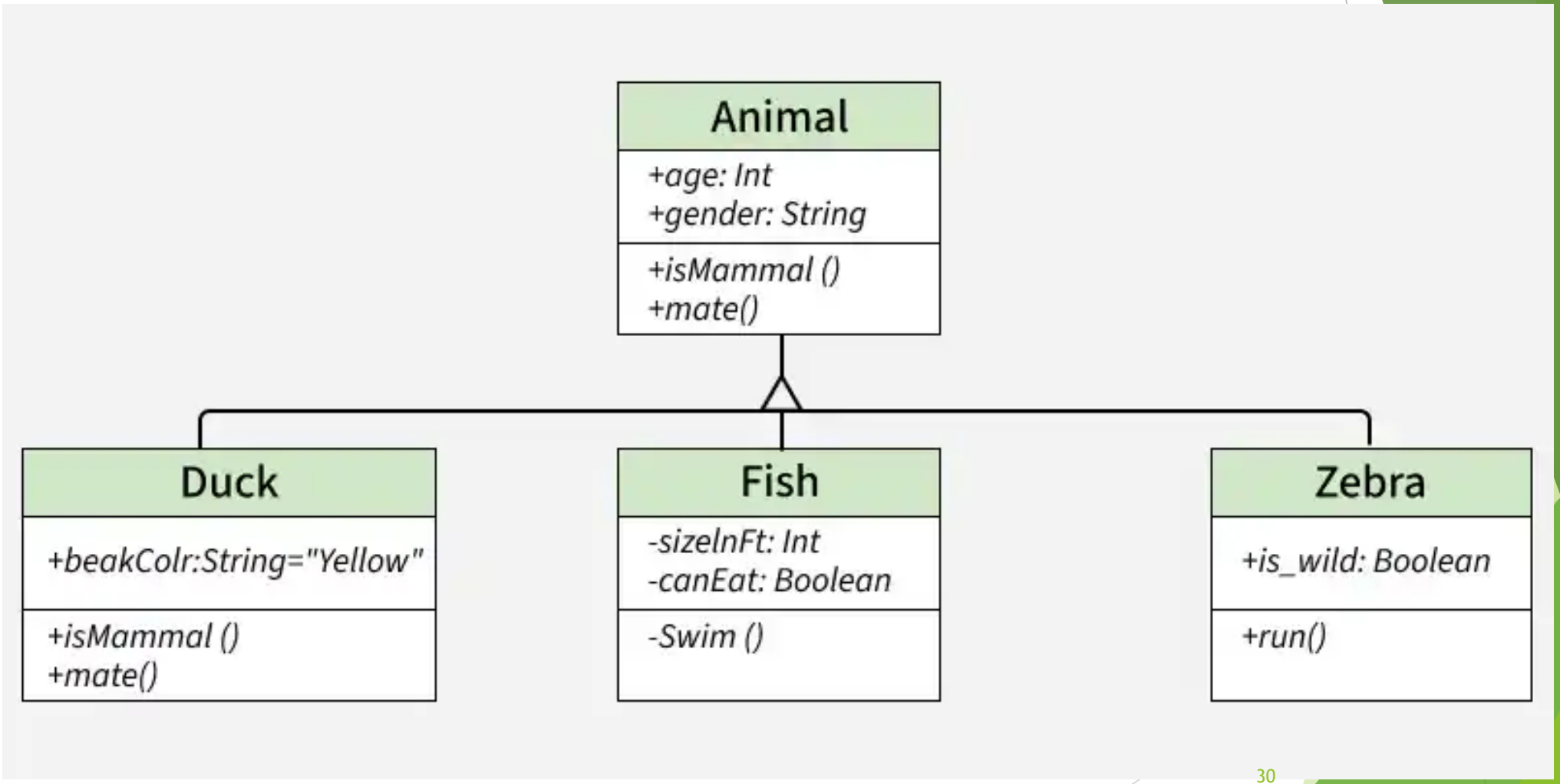
Behavior representation diagrams

- Use Case Diagram
- Activity Diagram
- State Machine Diagram
- Sequence Diagram
- Communication Diagram
- Interaction Overview Diagram
- Timing Diagram

1. Class Diagram

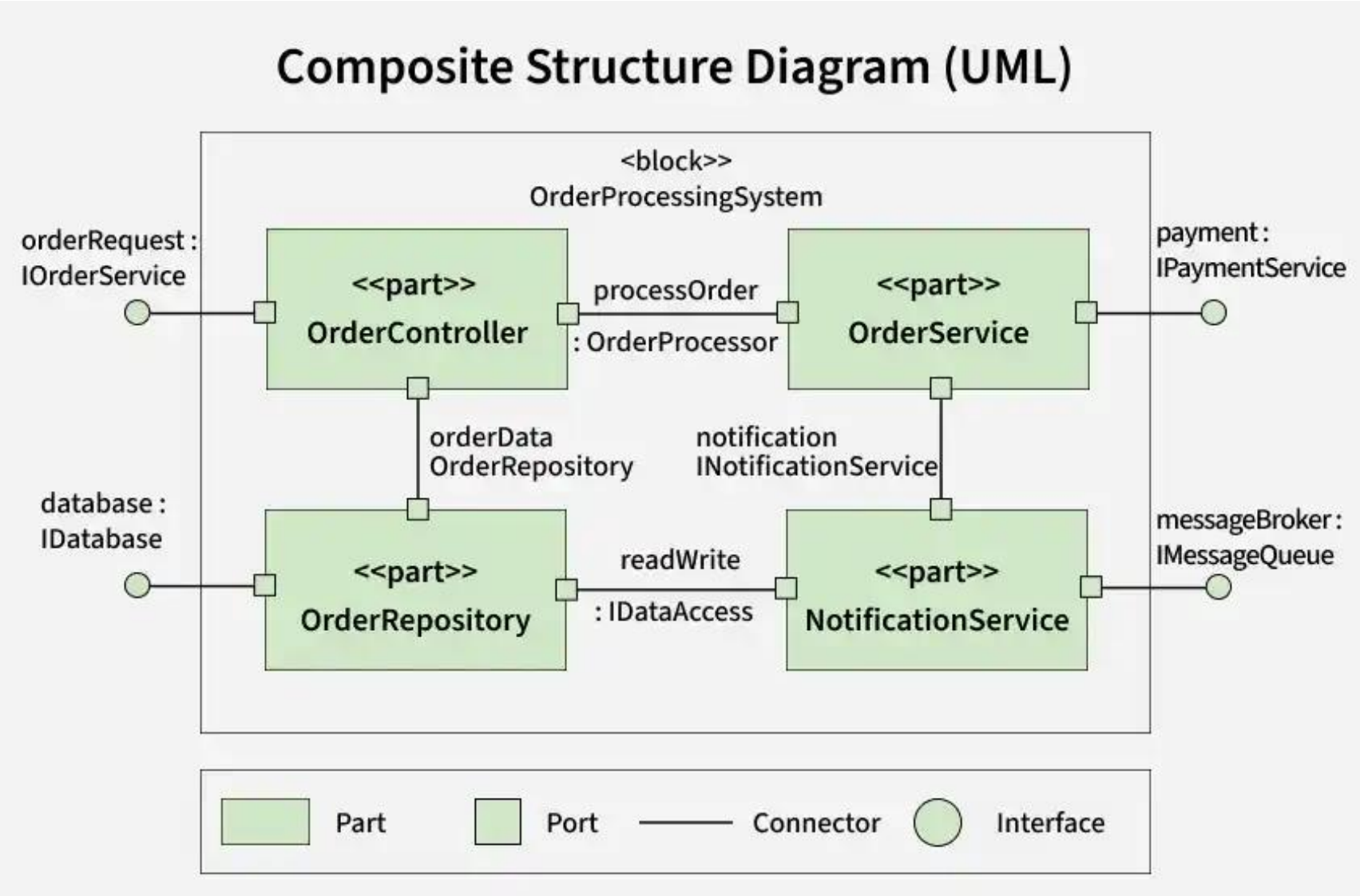
- A class diagram is the most widely used UML diagram and a fundamental part of object-oriented software design. It represents the static structure of a system by showing its classes, attributes, methods, and relationships between classes.
  - Class diagrams help visualize the structure of a system and its classes.
  - They show relationships between different classes, such as association, inheritance, and dependency.

Example


Animal
+age: Int
+gender: String
+isMammal ()
+mate()
Duck
+beakColr:String="Yellow"
+isMammal ()
+mate()
Fish
-sizeInFt: Int
-canEat: Boolean
-Swim ()
Zebra
+is_wild: Boolean
+run()

2. Composite Structure Diagram

- A composite structure diagram represents the internal structure of a class or other structured classifier and shows how its internal parts interact with each other. It provides a detailed view of the parts, ports, and connectors that make up the classifier.
  - It shows the internal parts and their relationships, including ports and connectors.
  - It provides a detailed view of a classifier's internal structure, unlike a class diagram, which focuses on the overall class.



Example: *Consider an Order Processing System where OrderController sends orders to OrderService, which uses OrderRepository to access the database and NotificationService to send notifications. Thus, the diagram shows the internal parts of the system and how they interact with each other.*

3. Object Diagram

- An object diagram represents a snapshot of a system at a particular point in time by showing instances of classes and the relationships between them. It helps visualize the actual state of objects after classes have been instantiated.
  - Object diagrams show specific instances of classes and their relationships.
  - They provide a snapshot of the system at a particular point in time, unlike class diagrams, which show the general class structure.

Example

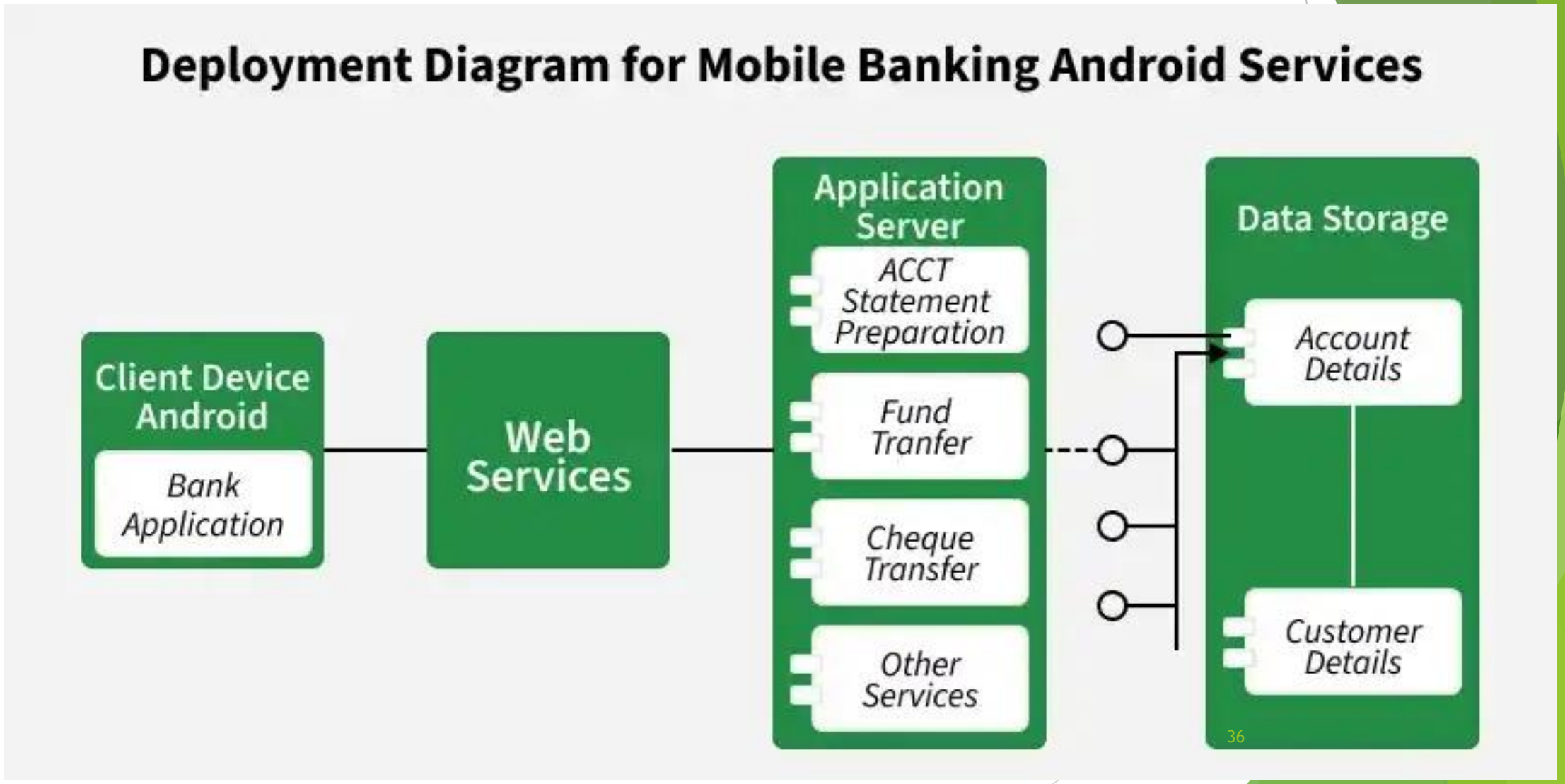


An object of class Student is linked to an object of class College.

4. Deployment Diagram

- A deployment diagram represents the physical arrangement of hardware and software in a system. It shows how software artifacts are deployed on and distributed across physical or virtual nodes.
  - Deployment diagrams show the distribution of software artifacts across hardware or execution environments.
  - They are useful for systems deployed across multiple machines with different configurations.

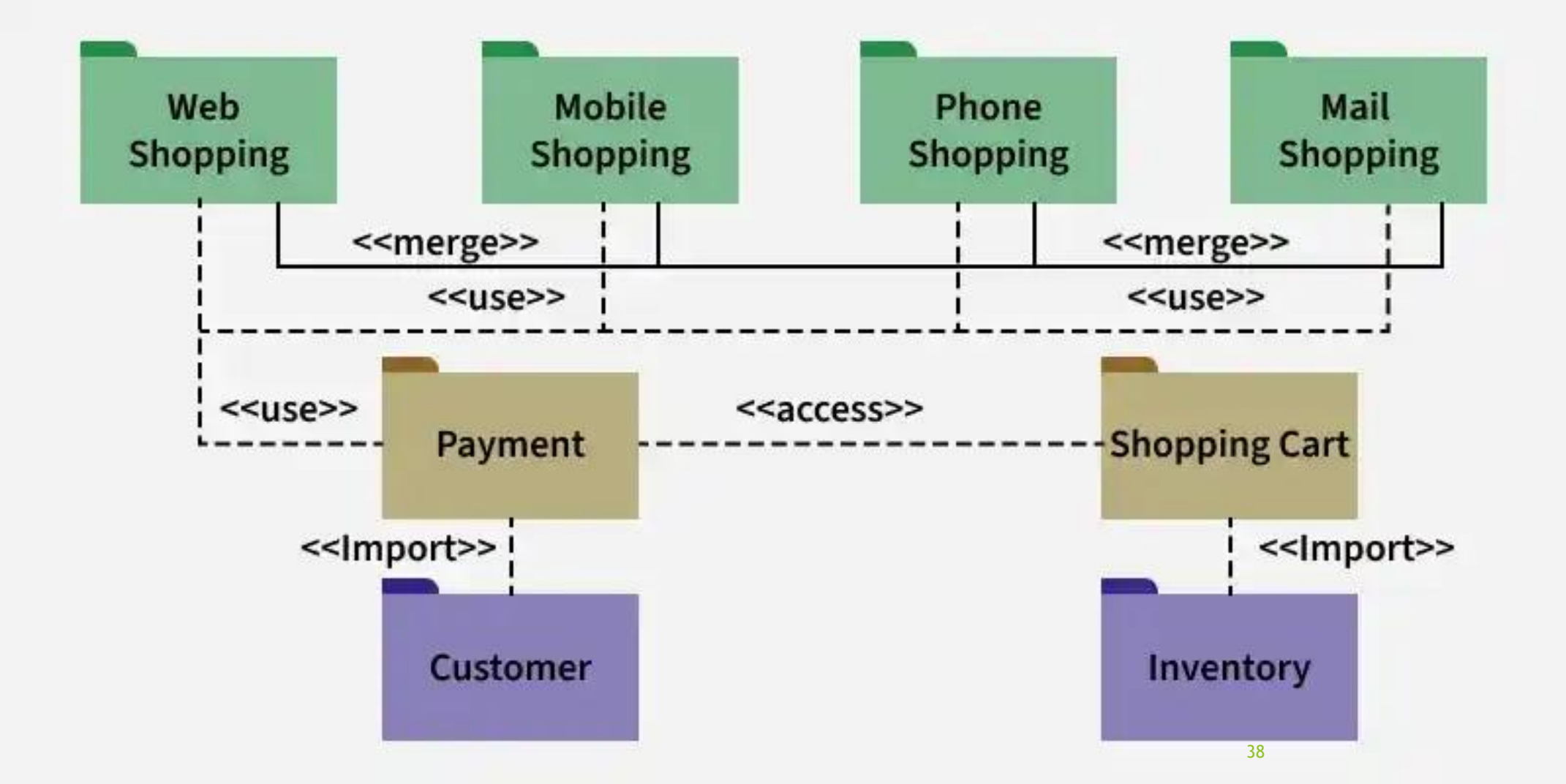
Example


Deployment Diagram for Mobile Banking Android Services
Client Device Android
Bank Application
Web Services
Application Server
ACCT Statement Preparation
Fund Tranfer
Cheque Transfer
Other Services
Data Storage
Account Details
Customer Details

5. Package Diagram

- We use Package Diagrams to depict how packages and their elements have been organized. A package diagram simply shows us the dependencies between different packages and internal composition of packages.
  - Packages help us to organize UML diagrams into meaningful groups and make the diagram easy to understand.
  - They are primarily used to organize class and use case diagrams.

Example


Web Shopping
Mobile Shopping
Phone Shopping
Mail Shopping
<<merge>>
<<merge>>
<<use>>
<<use>>
<<use>>
Payment
<<access>>
Shopping Cart
<<Import>>
<<Import>>
Customer
Inventory

Components Diagram

- A component diagram is a collection of vertices and arcs and commonly contain:
  - components
  - Interfaces and dependency
  - Aggregation
  - Generalization
  - Association
  - Realization relationships.
  - Notes and constraints.

CONT.: Component Diagram

- Component: A deployable unit of code within the system.
- Component Diagram: A diagram that shows relations between various Components and Interfaces.
- A component diagram breaks down the actual system under development into various high levels of functionality.
- Each component is responsible for one clear aim within the entire system and only interacts with other essential elements on a need-to-know basis.

Cont.: Component Diagram

- A component diagram represents the organization and dependencies of the physical or replaceable components of a software system. It helps visualize how components are structured and how they communicate through interfaces.
  - Component diagrams show the **structural relationships and dependencies** between system components.
  - They are useful for **complex and distributed systems**, including microservices, to represent service boundaries, APIs, and communication.

MODELLING COMPONENTS IN UML

- Unified Modeling Language, is a standardized modeling language consisting of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems.
- The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

USING UML

- In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components or software systems.
- It's used to represent the architectures of the software components and the dependencies between them.
- Such as those software components including run-time components, executable components also the source code components.

IMPORTANCE OF UML

- The UML is a very important part of developing object-oriented software and the software development process.
- The UML uses mostly graphical notations to express the design of software projects.
- Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software. In this article, we will give you detailed ideas about what is UML, the history of UML and a description of each UML diagram type, along with UML examples.

Basic Concepts of Component Diagram

(i) A component : represents a modular part of a system that encapsulates its contents and whose manifestation is replaceable within its environment.

- In UML 2, a component is drawn as a rectangle with optional compartments stacked vertically. It can be represented
  - A rectangle with the component's name
  - A rectangle with the component icon
  - A rectangle with the stereotype text and/or icon

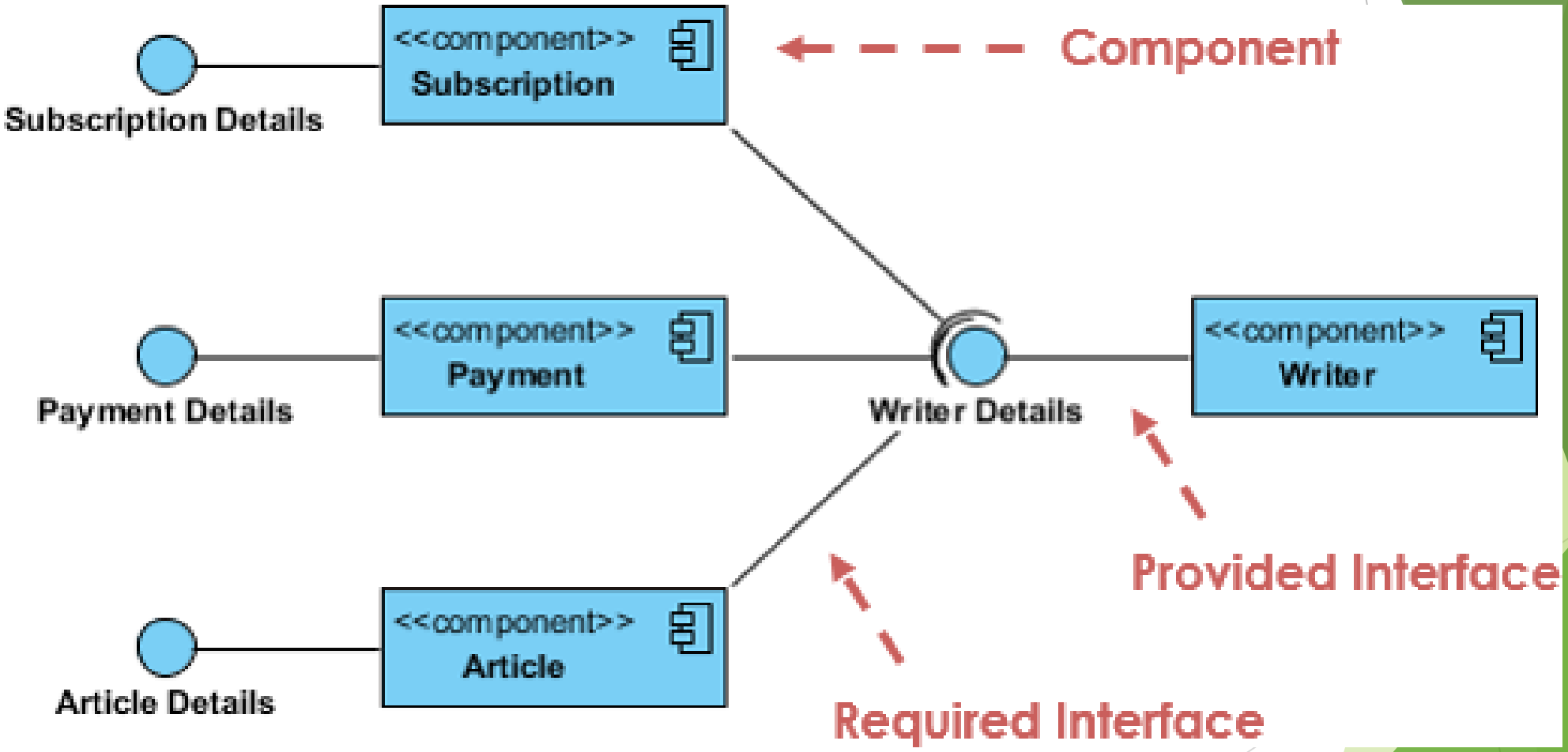


Syntax

Component in Component-Based Diagram

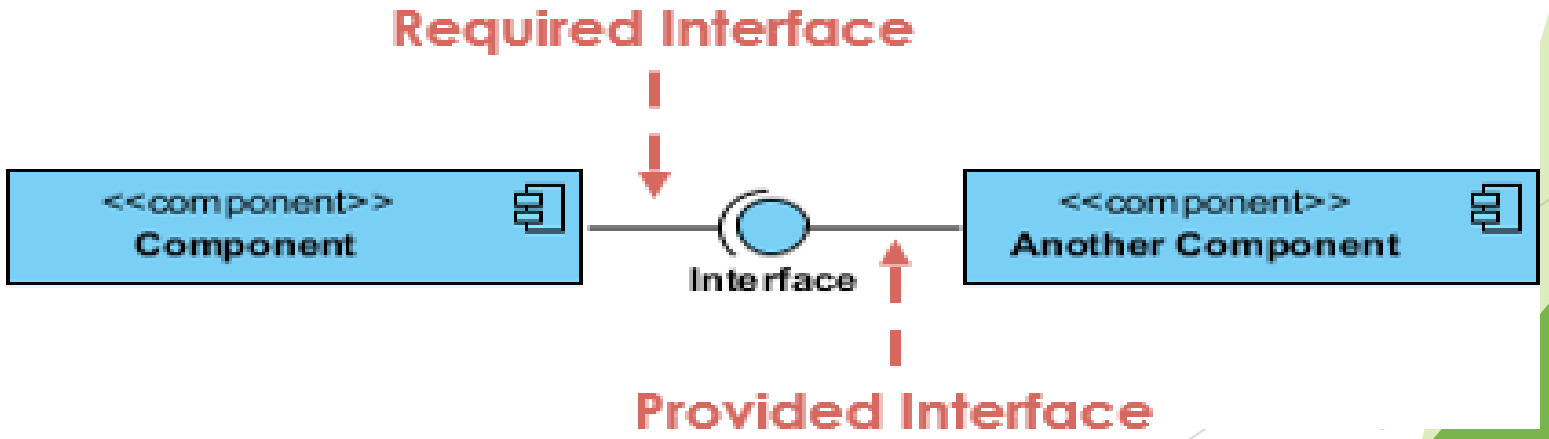


EXAMPLE

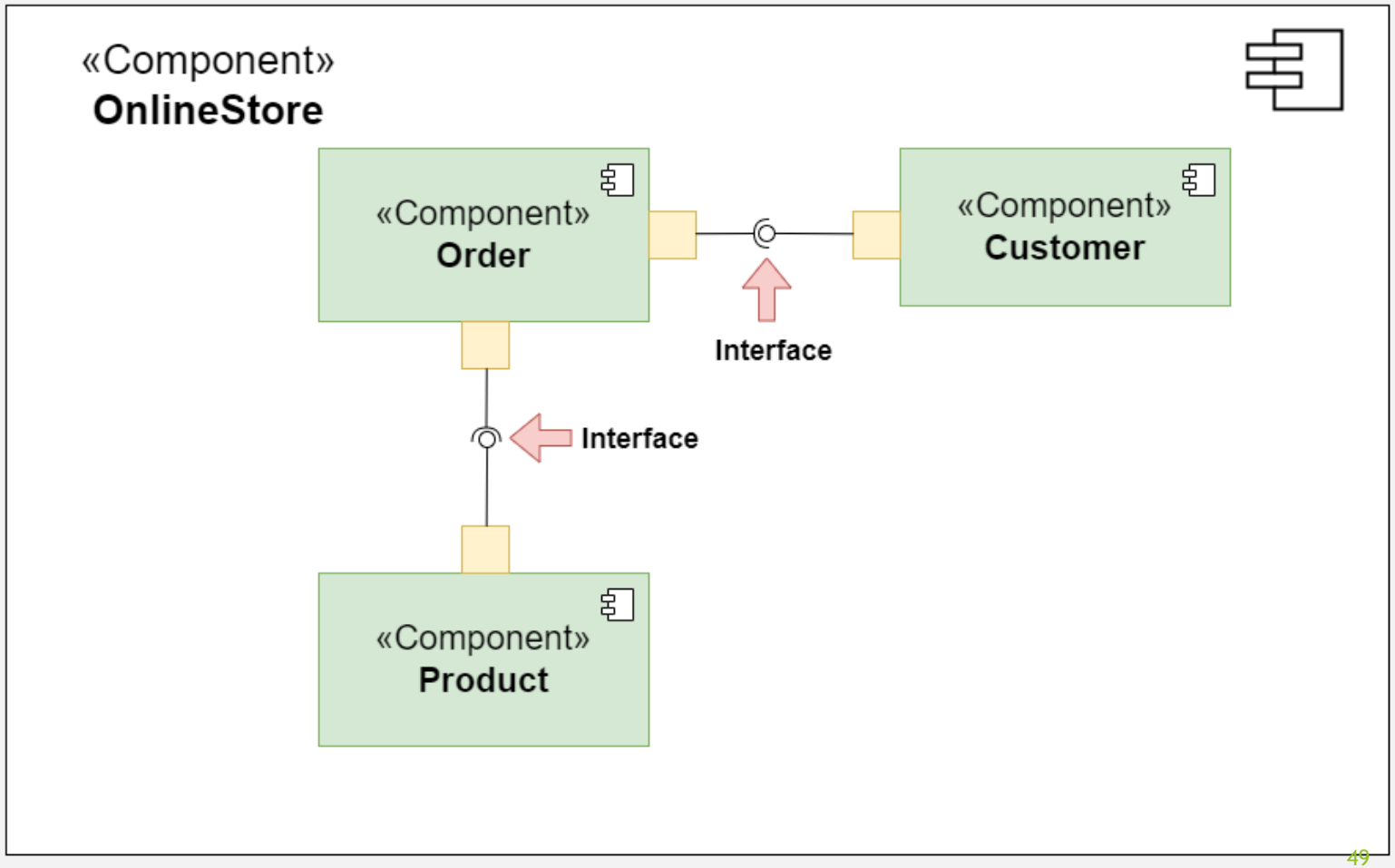

<<component>>
Subscription
Component
Subscription Details
<<component>>
Payment
Payment Details
Writer Details
<<component>>
Writer
Provided Interface
<<component>>
Article
Article Details
Required Interface

(ii) Interface

- **Provided interface** symbols with a complete circle at their end represent an interface that the component provides - this "lollipop" symbol is shorthand for a realization relationship of an interface classifier.
- **Required Interface** symbols with only a half circle at their end (a.k.a. sockets) represent an interface that the component requires (in both cases, the interface's name is placed near the interface symbol itself).
- **Function:** Define how components communicate with each other, ensuring that components can be developed and maintained independently.



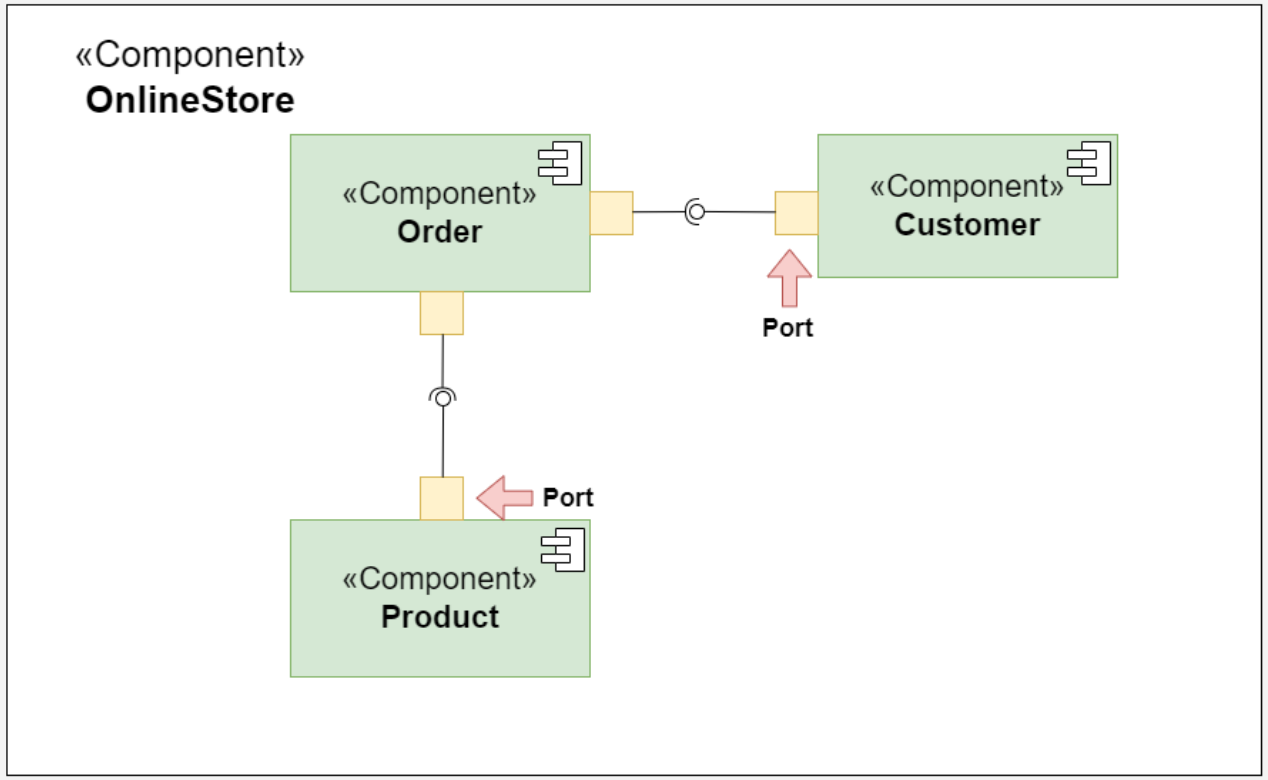
Interfaces in Component-Based Diagram

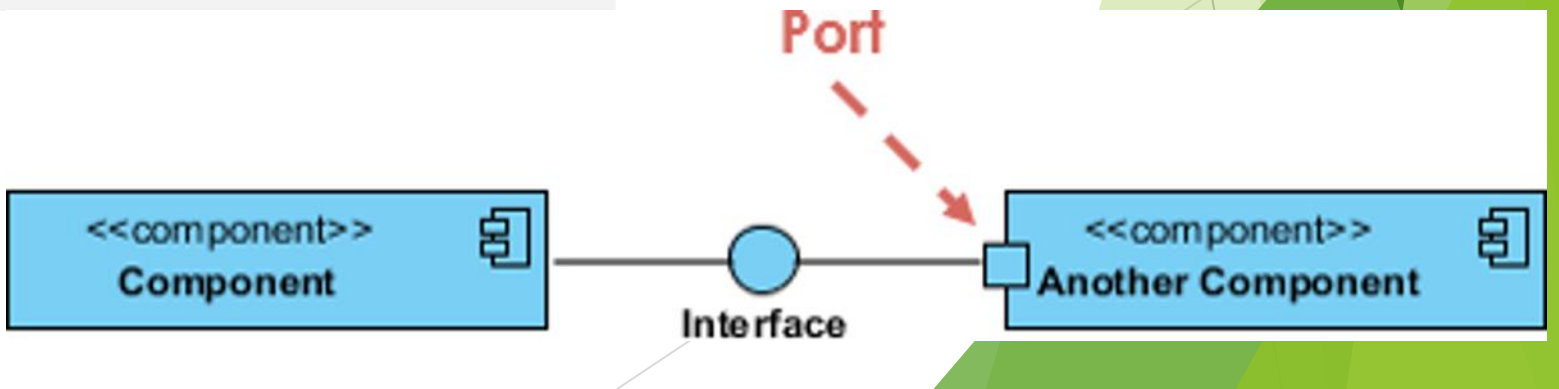


(iii) PORT

- Ports are represented using a square along the edge of the system or a component.
- A port is often used to help expose required and provided interfaces of a component.
- **Role:** Represent specific interaction points on the boundary of a component where interfaces are provided or required.
- **Function:** Allow for more precise specification of interaction points, facilitating detailed design and implementation.

Ports in Component-Based Diagram

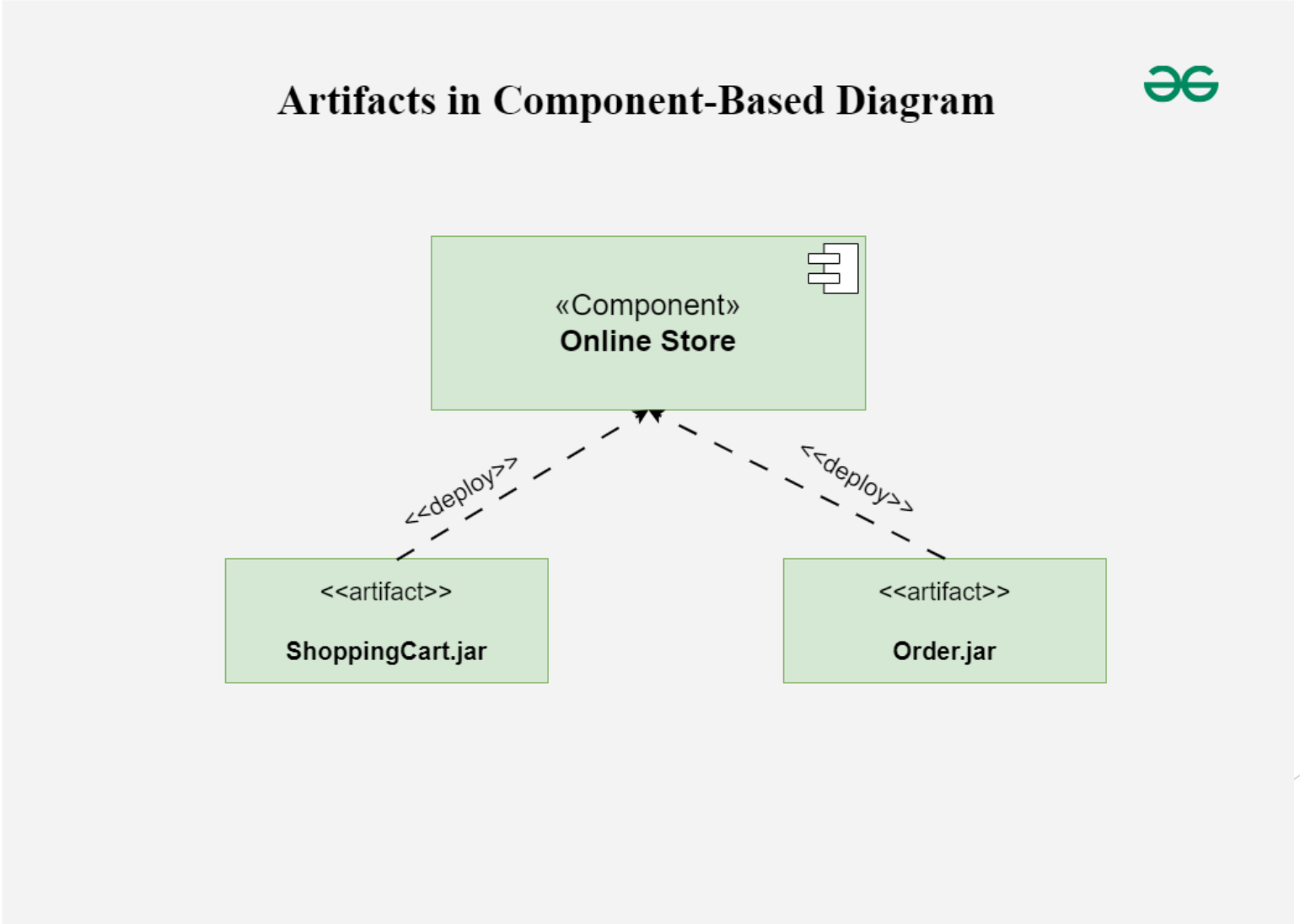




(iv). Artifacts

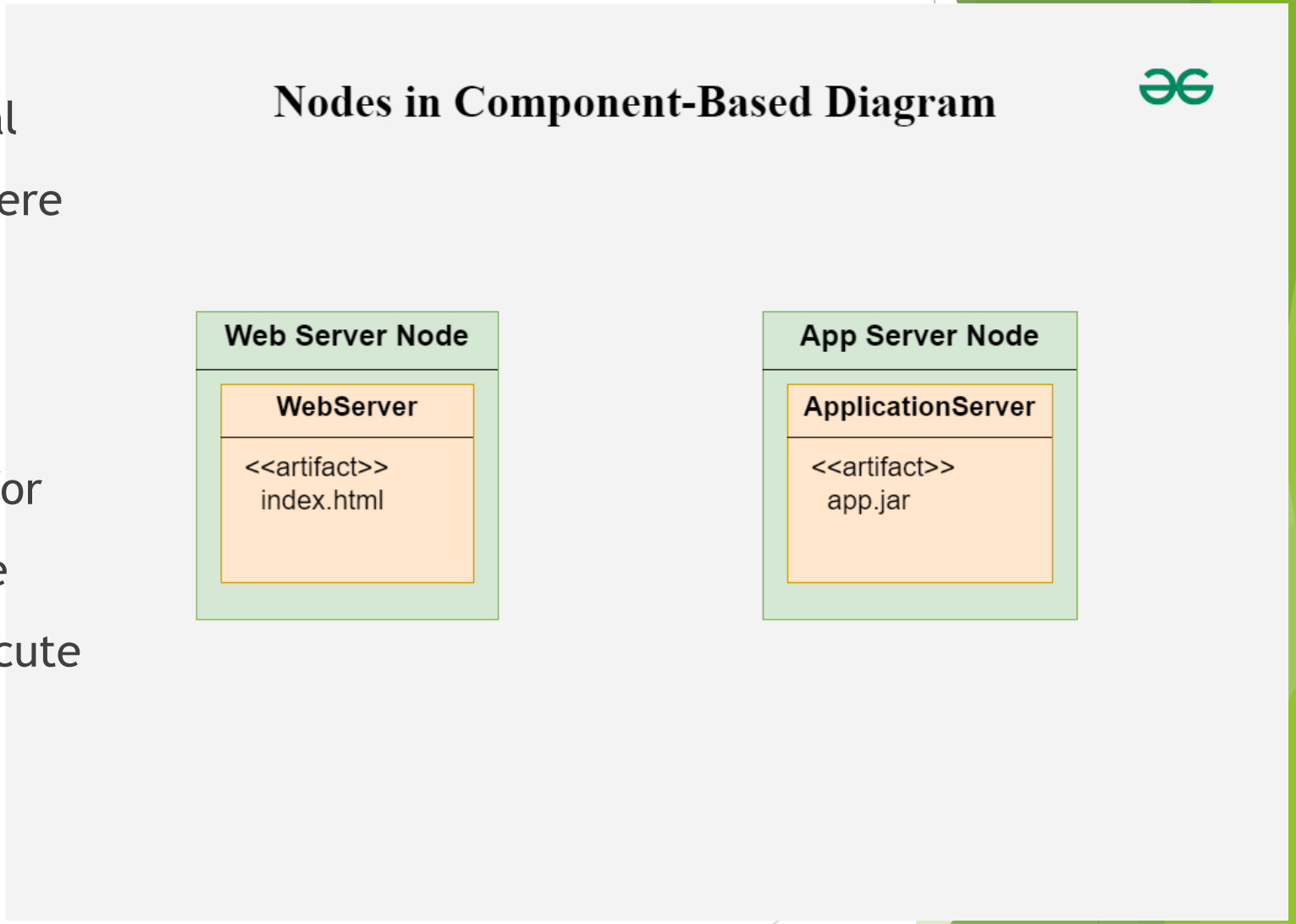
- Represent physical files or data that are deployed on nodes.
  - **Symbol:** Rectangles with the artifact stereotype («artifact»).
  - **Function:** Show how software artifacts, like executables or data files, relate to the components.

Example- Artifacts



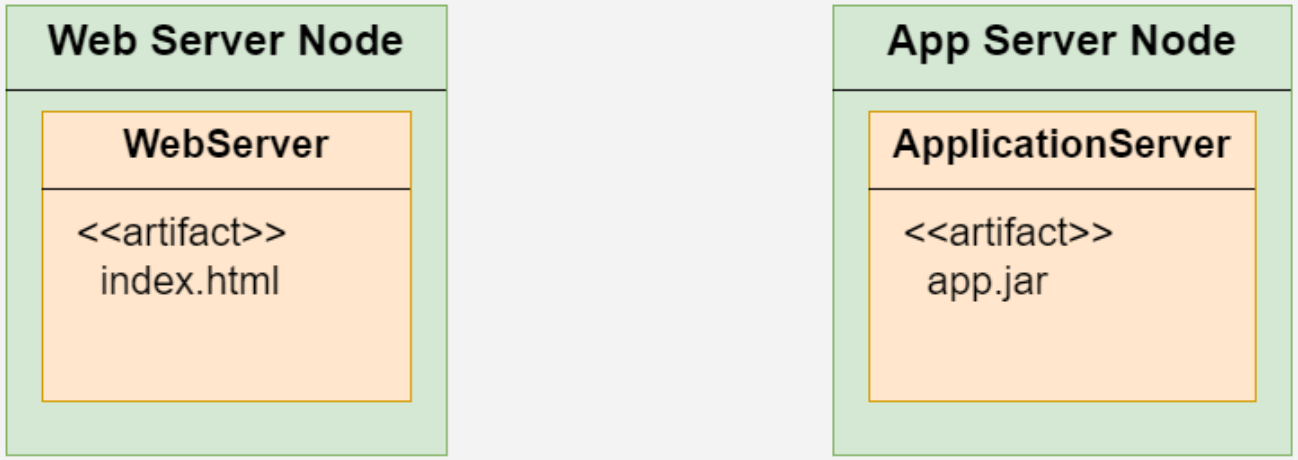
(v) Nodes

- Represent physical or virtual execution environments where components are deployed.
- **Symbol:** 3D boxes.
- **Function:** Provide context for deployment, showing where components reside and execute within the system's infrastructure.



Node

Nodes in Component-Based Diagram



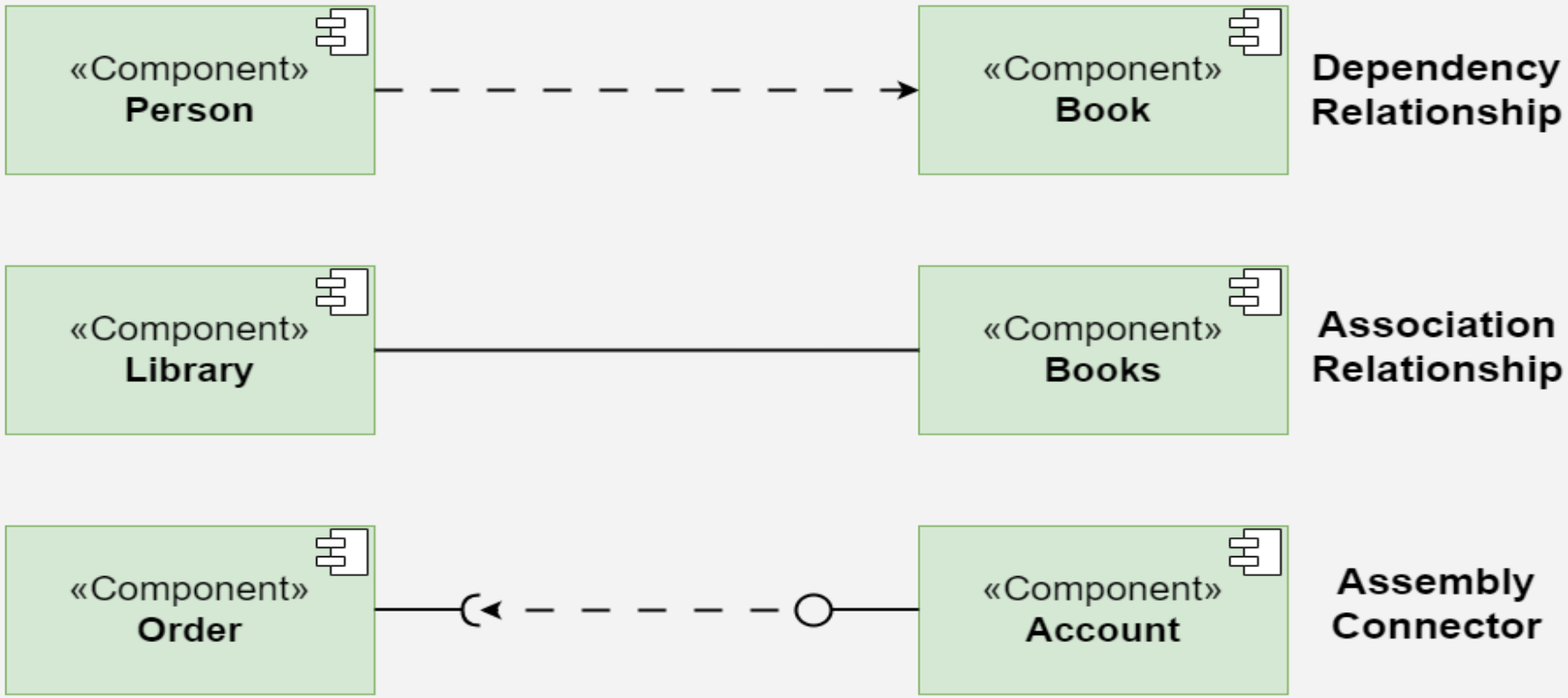
(vi) RELATIONSHIPS

- Graphically, a component diagram is a collection of vertices and arcs and commonly contain components, interfaces and dependency, aggregation, constraint, generalization, association, and realization relationships.
- It may also contain notes and constraints

Cont.: Relationships

- Relationships represent the connections and dependencies between components and interfaces, showing how different parts of the system interact.
- **Dependency (Dashed Arrow):** Indicates that one component relies on another component or interface to perform its functionality.
- **Association & Assembly Connector:** A solid line shows a direct relationship between components, while an assembly connector links a required interface of one component to a provided interface of another.
- **Function:** Helps visualize communication paths, dependencies, and interactions between components within the system architecture.

Relationships in Component-Based Diagram



ASSOCIATION

- An association specifies a semantic relationship that can occur between typed instances.
- It has at least two ends represented by properties, each of which is connected to the type of the end.
- More than one end of the association may have the same type.
- NOTATION _____

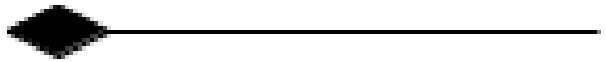
Aggregation

- A kind of association that has one of its end marked shared as kind of aggregation, meaning that it has a shared aggregation.



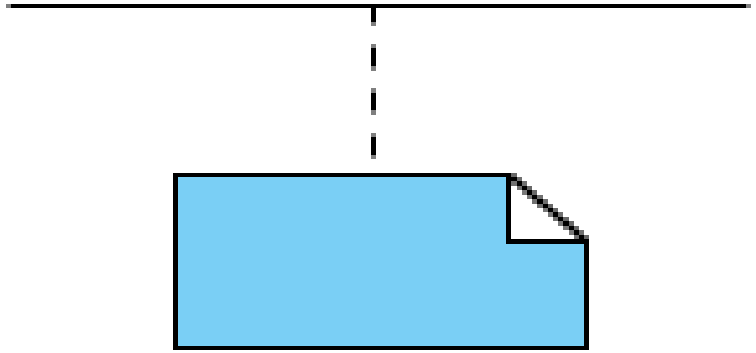
Composition

- Composite aggregation is a strong form of aggregation that requires a part instance be included in at most one composite at a time.
- If a composite is deleted, all of its parts are normally deleted with it.
- NOTATION



Constraint

- A condition or restriction expressed in natural language text or in a machine readable language for the purpose of declaring some of the semantics of an element.



Dependency

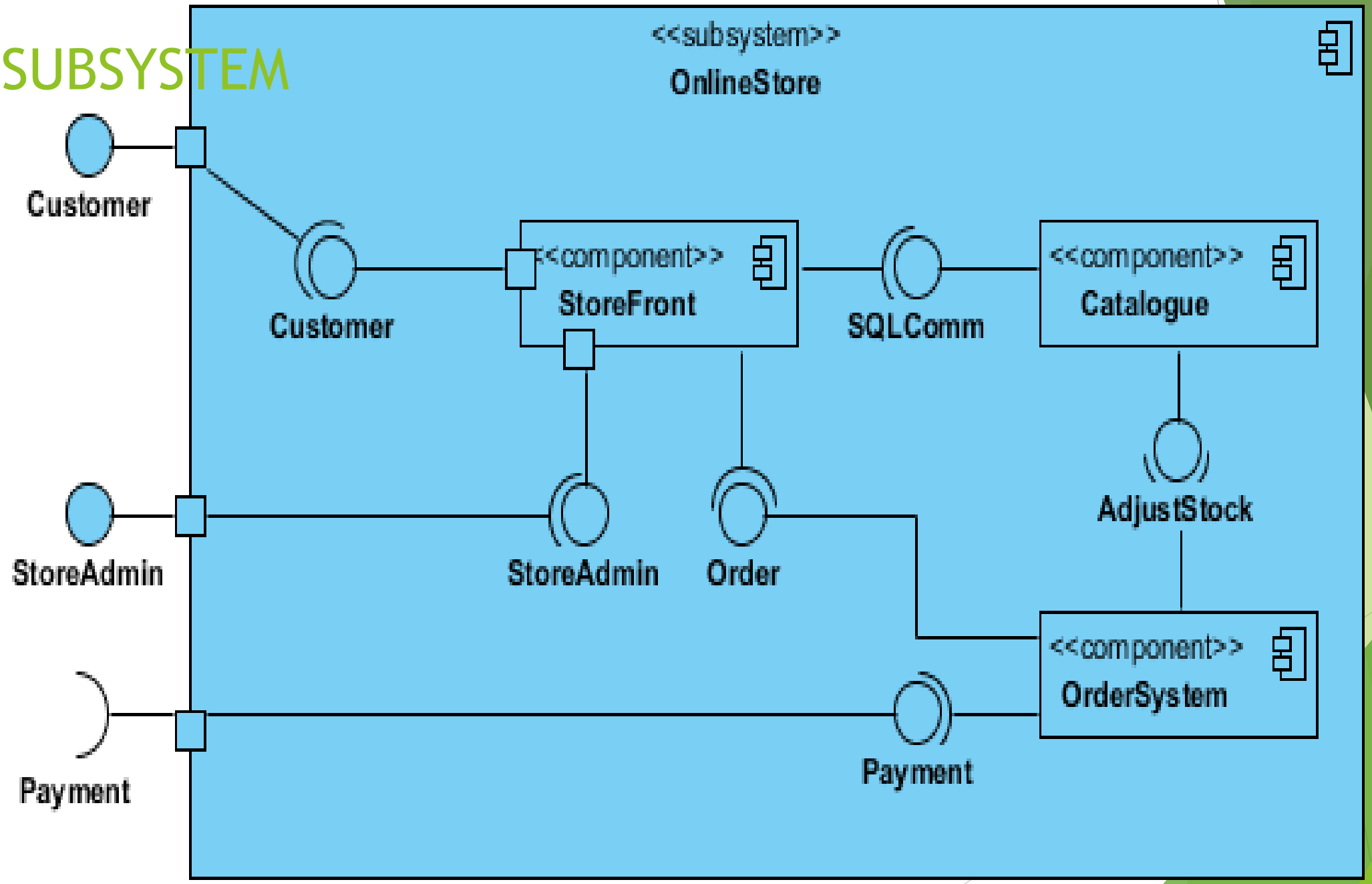
- A dependency is a relationship that signifies that a single or a set of model elements requires other model elements for their specification or implementation.
- This means that the complete semantics of the depending elements is either semantically or structurally dependent on the definition of the supplier element(s).



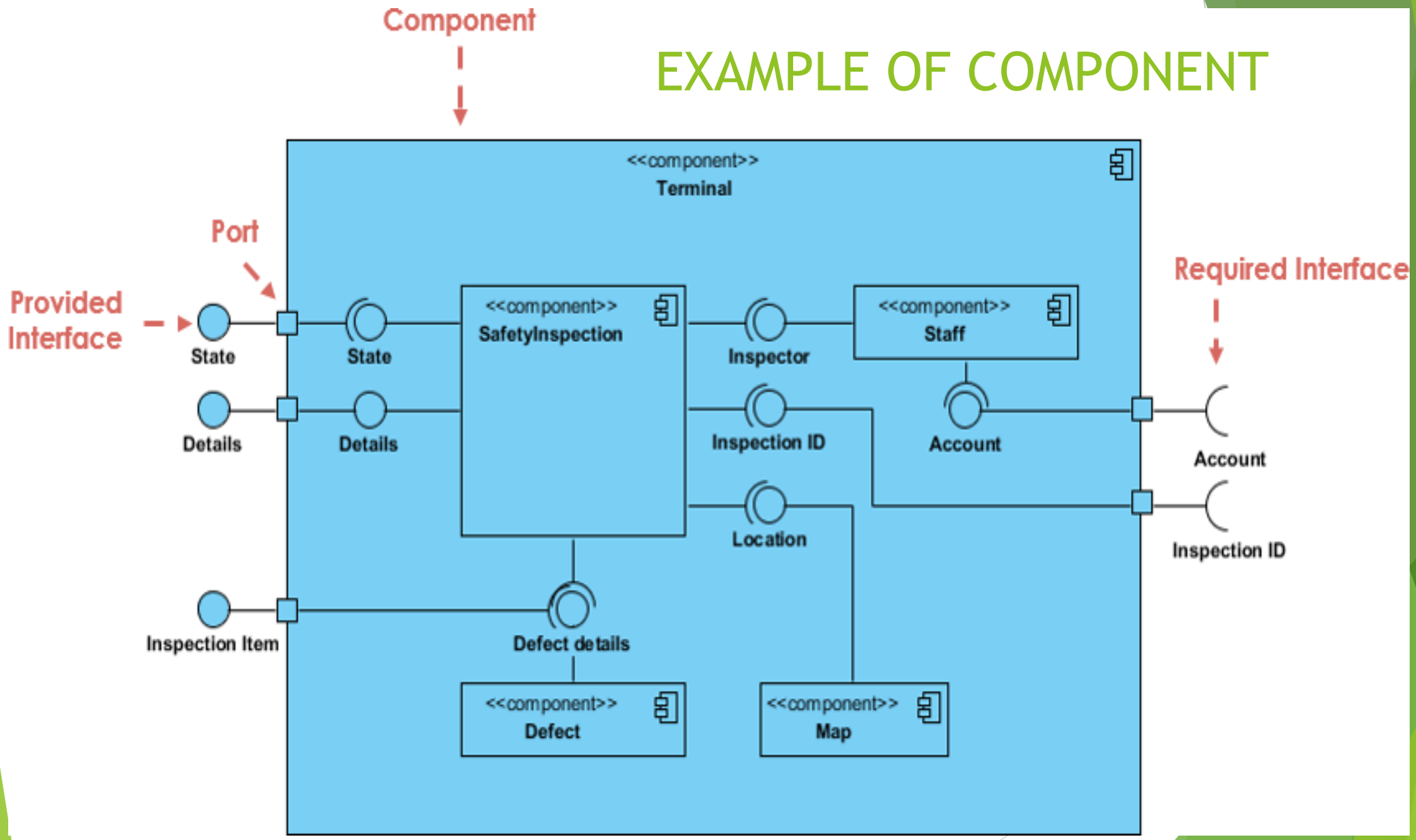
Subsystems

- The subsystem classifier is a specialized version of a component classifier.
- Because of this, the subsystem notation element inherits all the same rules as the component notation element.
- The only difference is that a subsystem notation element has the keyword of subsystem instead of component. EXAMPLE

SUBSYSTEM


<<subsystem>>
OnlineStore
Customer
Customer
<<component>>
StoreFront
SQLComm
<<component>>
Catalogue
AdjustStock
StoreAdmin
StoreAdmin
Order
<<component>>
OrderSystem
Payment
Payment

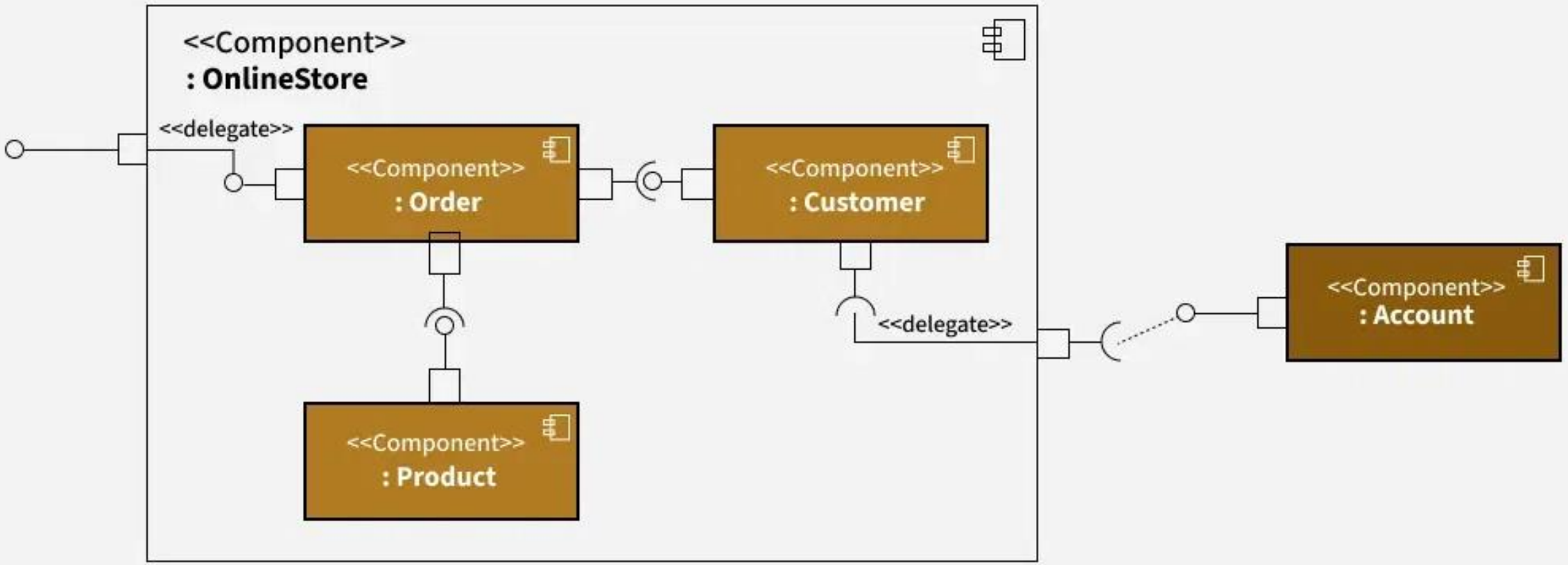
EXAMPLE OF COMPONENT



Example of Component Based Diagram

This component diagram represents an **Online Store** system, breaking it down into various functional components and showing how they interact. Here's a breakdown of each part:

Component based Diagram Example



Explanation

1. OnlineStore Component : Represents the main system and encapsulates the core functionality of the online store. It contains the Order, Customer, and Product components.

2. Order Component: Handles order-related operations within the OnlineStore system.

- Connected to the Product component to access product information for orders.
- Connected to the Customer component to associate orders with customers.
- Uses delegate connectors (<>) to route certain internal operations to other components or systems

3. Customer Component: Manages customer data and customer-related activities.

- Connected to the Order component for processing and managing customer orders.
- Linked to the Account component through a delegate connection, allowing access to account-related information from an external system.

Cont.:

4. Product Component: Manages product information and product-related operations.

- Connected to the Order component so orders can reference and use available products.

5. Account Component: Represents an external component located outside the OnlineStore boundary.

- Connected to the Customer component through a delegate relationship.
- Handles account-related functions that are delegated from the OnlineStore system.

Applications of Component-Based Diagrams

Component-Based Diagrams help visualize system structure, improve communication, and support the development and maintenance of software systems.

- **System Design and Architecture:** Helps architects understand system structure, component interactions, and dependencies.
- **Requirements Analysis:** Assists stakeholders in understanding functional and non-functional system requirements.
- **System Documentation:** Serves as a reference for documenting architectural decisions and overall system design.
- **Software Development:** Guides developers by defining component boundaries and interactions.
- **Code Generation and Implementation:** Can be used as a foundation for generating code and accelerating development.
- **System Maintenance and Evolution:** Makes it easier to analyze, modify, and extend the system as requirements change.

Tools and Software available for Component-Based Diagrams

Several tools and software are available for creating Component-Based Diagrams, ranging from general-purpose diagramming tools to specialized UML modeling software. Here are some popular options:

1. **Lucidchart:** Lucidchart is a cloud-based diagramming tool that supports creating various types of diagrams, including Component-Based Diagrams.
2. **Microsoft Visio:** Microsoft Visio is a versatile diagramming tool that supports creating Component-Based Diagrams and other types of UML diagrams.
3. **Visual Paradigm:** Visual Paradigm is a comprehensive UML modeling tool that supports the creation of Component-Based Diagrams, along with other UML diagrams.
4. **Enterprise Architect:** Enterprise Architect is a powerful UML modeling and design tool used for creating Component-Based Diagrams and other software engineering diagrams.
5. **IBM Rational Software Architect:** IBM Rational Software Architect is an integrated development environment (IDE) for modeling, designing, and developing software systems.

End

!!!!!!!